

Understanding REACT HOOKS



Agenda



- React and its component structure
- What are Hooks?
- React Hooks and their capabilities
- Migrating Your Existing Apps to React Hooks
- Combine Existing React Hooks into New Custom Hooks
- Benefits of using React Hooks
- Best Practices



The diagram illustrates the structure of React components, organized into a grid of six colored boxes, each containing a React logo icon and a number. The boxes are arranged in two rows and three columns. The top row contains boxes 1 (blue), 2 (orange), and 3 (green). The bottom row contains boxes 5 (blue), 6 (orange), and 4 (green). Box 5 is the largest, spanning the width of boxes 1 and 2. Box 4 is the widest, spanning the width of boxes 2 and 3. The text 'React Components' is centered over the grid.

React Components

React and its component structure

- The UI of React applications is made up of components.
- React provides the capability to define components in two ways:
 - Functional Components

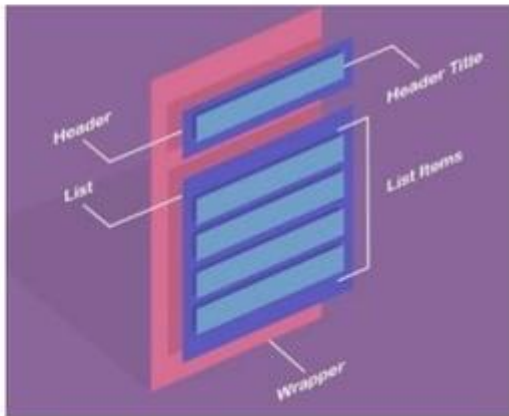
```
function Welcome(props) {
  return <h1>Hello, {props.name} !</h1>;
}
```

OR

```
const element = <Welcome name="WTT" />;
```

- Class Components

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```



React and its component structure(CONTD.)

- Before the arrival of React Hooks, the main difference between the Functional and Class components was
 - Class components have the Capability of handling local state at Component level
 - Class components have lifecycle methods associated with events of the component life which can be implemented as needed to tap the event and add some functionality. For. e.g. `componentDidMount`(after mounting), `componentDidUpdate`(after update), `componentWillUnmount`(before unmounting)
- Functional components were only for presentation purpose as they could not handle any state of their own and had no special methods.



React Hooks

What are React Hooks?

"Hooks represent the vision of the future of React"

- React Hooks are functions that let us hook into the React state and lifecycle features for functional components.

React Hooks Introduction - useState

```
function SimpleForm(props) {  
  const [firstName, setFirstName] = useState('');  
  const [lastName, setLastName] = useState('');  
  const [age, setAge] = useState('');  
  
  // ...  
}
```

Why Hooks? (answers from the now-famous talk at React Conf 2018 as to what in React sucks)



- Reusing Logic
 - Before hooks, reuse was possible using Higher order components and render props, i.e. passing functions as props. This required restructuring your app, sometimes to include additional components.
- Giant Components
 - Huge components with multiple lifecycle methods handling different pieces of logic
- Confusing Classes
 - Can we really do without the concept of classes??
 - *Classes are hard for humans, but it's not just humans, classes are also hard for machines -- Sophie Alpert*
 - Binding Issues, Performance Issues, Strict hierarchies

React Hooks and their capabilities

- If any of the value in the array changes, the callback will be fired after every render,
- When it's not present, the callback will always be fired after every render.
- When it's an empty list, the callback will only be fired once, similar to componentDidMount.

useState

- Define state structure by giving initial value
- Returns a reference to current state value and a function(like setState) to set the new value for state

```
const [ users, setUsers]= useState([]);
```

example

useEffect

- This is an equivalent for class lifecycle methods (componentDidMount,componentDidUpdate,componentWillUnMount)
- Can be controlled using the second dependency parameter

```
useEffect(()=>{  
  //Logic to handle  
},secondParameter);  
secondParameter will decide how many  
times the logic is invoked
```

useRef

- useRef is used to get a reference to the DOM or any other element which can be referred and manipulated directly

```
newRef = useRef("");
```

useReducer

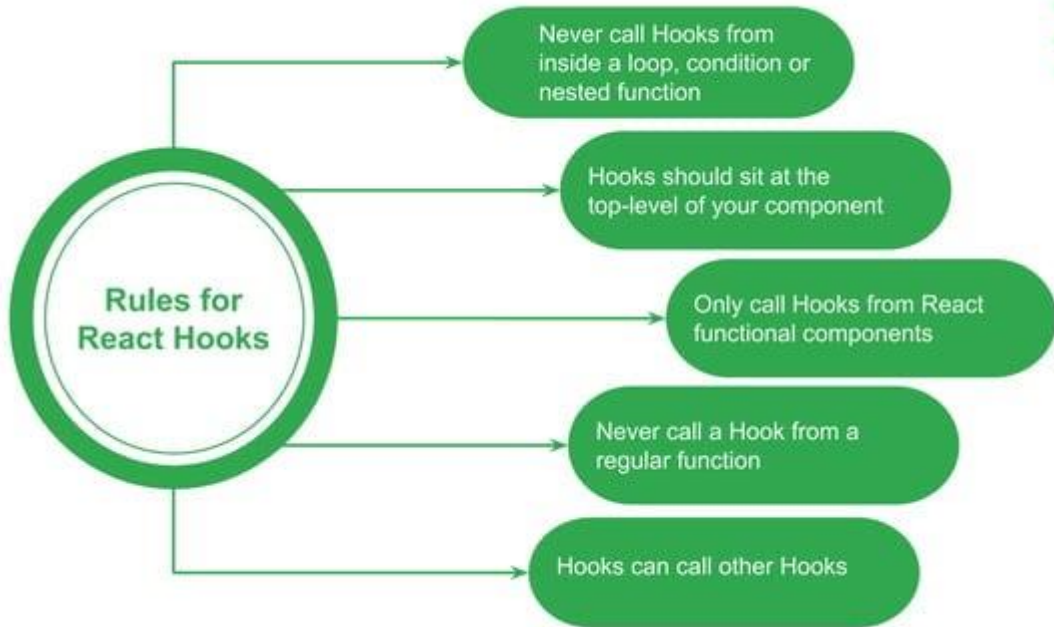
- Single hook which handles state initialization as well as actions on the state dispatched in the form of events.

```
Const[state,  
dispatch]=useReducer(reducerFunction,  
initialState);
```

example

React Hooks and their capabilities

 useContext	• To get the current value of a context object • Equivalent to a context Consumer • Simpler and cleaner syntax	<pre>const value = useContext(MyContext);</pre>
 useMemo	• Takes a function and an array of dependencies and returns a memoized value which will be recalculated when one of the dependencies change. • Used for performance optimization	<pre>const memoizedValue = useMemo(() => computeValue(a, b), [a, b]);</pre>
 useCallback	• Takes an inline callback and an array of dependencies and returns a memoized callback which changes only if any of the dependencies change. • Used for optimization while passing the callback as a prop to avoid unnecessary render.	<pre>const memoizedCallback = useCallback() { doSomeLogic(a, b); }, [a, b];</pre>



Migrating Your Existing Apps to React Hooks

- React official does recommend to start using functional components as the preferred way to define components due to advantages they provide.
- In order to implement hooks in your existing apps, you can follow the steps:

Convert Class components to Functional components(Remove this keyword wherever used. No more classes and no more this!)

The existing state can be converted to single or multiple pieces of state depending on the usage. Always remember to initialise state.

You can make use of `useState` to handle state definition and initialization.

You can replace the lifecycle methods(`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`) logic with the single `useEffect` hook.

You can segregate all the hook related logic into a custom hook and use across components.(We will see how this can be done in next slide)

You can use `useReducer` hook to handle different actions on state at one place.

Combine Existing React Hooks into New Custom Hooks

- React provides the capability of defining custom hooks which can be defined and used just like the predefined react hooks.
- Custom hooks are functions used for accessing and manipulating state with use of standard React hooks.
- All the hook related logic using the standard hooks can be defined in a separate js file and named with a use prefix to indicate that it is a custom hook.
- This can be imported like a standard hook in any component and used to get the entire state logic available for reuse.
- It is not the same copy of state when the custom hook is reused and every component which uses it, gets its own local copy of the state.
- It provides for capability of defining the logic once and using it as many number of times as needed.
Example - <https://codesandbox.io/s/react-custom-hooks-demo-qtrr0>
- This capability has enabled the React community to come up with many new hooks providing reusable logic. This website provides a consolidated collection - <https://nikgraf.github.io/react-hooks/>

Benefits of using React Hooks

1

The functional component code is cleaner and easier to understand.

2

Hooks let you split state logic into smaller functions, based on which pieces are related.

3

Component State can also be split as related pieces put together.

4

Hooks are fast. Some performance tests conducted on class vs functional components performed on simple counter component showed the functional component using hooks to be faster than class components by 3%. Also hooks may work better with future React optimizations (like ahead of time compilation and components folding).

5

Custom hooks can be created enabling to reuse the entire state logic available which is not possible in class components.

Best Practices



Best Practices

- Write small and simple hooks.
- Don't break related state into separate pieces such that update in one triggers an update of the other causing unnecessary renders.
- Don't create custom hooks just for the sake of it. Also while returning values in custom hooks make sure you return an updated value only if it really changed to avoid unnecessary render.
- Make use of libraries like why-did-you-render library that tells you about avoidable re-renders. To use it, mark your component as `TestComponent.whyDidYouRender = true`, start using it, and look for messages in the console showing if it is being rerendered unnecessarily.
- Keep the hooks logic organized and in the normal conventional lifecycle flow to make the code easily understandable.



Questions and Answers